Assignment 2: Big Data Storage & Retrieval

Comparative Implementation and Evaluation using PostgreSQL, MongoDB, and Neo4j

TIVDZUA LUBEM NOAH

March 2026

Abstract

This report presents the design, implementation, analysis, and benchmarking of three database solutions for an e-commerce dataset: PostgreSQL as the relational model, MongoDB as the document model, and Neo4j as the graph model. The selected dataset snapshot contains event logs, campaign messages, friendship links, campaign metadata, and first-purchase information.

The objective of this project was to design appropriate data models for each database system, clean and load the dataset, execute analytical queries, and compare query performance across the systems.

The first analysis studied whether marketing campaigns influenced customers to purchase products. Purchase-rate rankings were consistent across PostgreSQL, MongoDB, and Neo4j, showing that campaigns generated measurable conversion activity. The second analysis produced personalized product recommendations using friendship relationships and weighted behavioral events. The third analysis retrieved recommended products through a final retrieval stage.

Benchmarking experiments were conducted by executing each query five times per database and calculating mean execution time and standard deviation. PostgreSQL produced the fastest and most stable execution for analytical queries, while MongoDB provided flexibility for data ingestion and Neo4j offered a natural representation of social relationships. The results demonstrate that database technologies perform differently depending on workload characteristics and that hybrid architectures can provide the most effective solution for large analytical systems.

1 Data Modeling

1.1 Dataset and Preprocessing

The project used an e-commerce dataset containing event logs, campaign messages, social connections between users, and campaign metadata.

The selected dataset snapshot contained the following source files:

- events.csv
- campaigns.csv

- messages.csv
- client_first_purchase_date.csv
- friends.csv

A Python-based cleaning pipeline was implemented to normalize timestamps, convert identifiers to numeric values, remove duplicate friendships, and generate stable event identifiers. The preprocessing stage also corrected boolean field parsing issues detected during MongoDB validation.

All database diagrams were created using Hackolade and exported for inclusion in the report.

1.2 PostgreSQL Data Model

The PostgreSQL implementation follows a normalized relational schema consisting of dimension and fact tables designed to support analytical queries.

- dim_campaigns
- dim_clients
- dim_products
- dim_users
- fact_events
- fact_messages
- fact_friendships

Dimension tables store stable entities such as users, campaigns, and products, while fact tables store behavioral interactions and messaging activity. This design allows efficient aggregation and joins across large datasets.

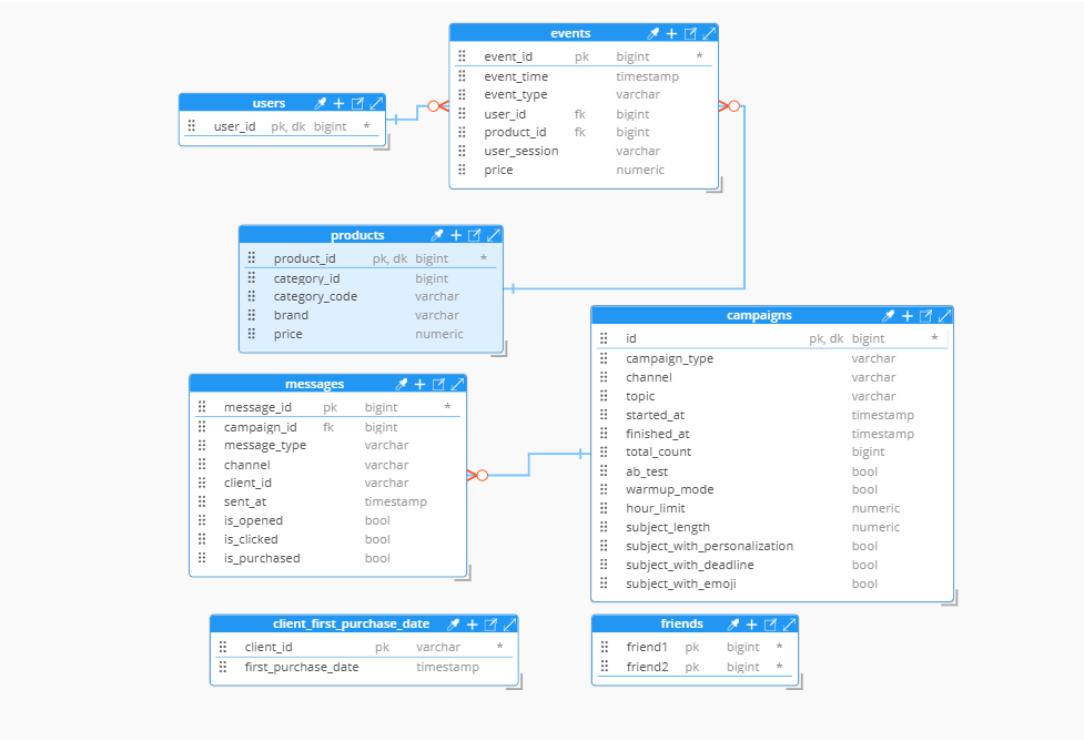

Figure 1: PostgreSQL relational data model created in Hackolade.

1.3 MongoDB Data Model

MongoDB stores the dataset as collections representing campaigns, events, messages, client_first_purchase_date, and friendships.

Large datasets were imported using chunked JSON batches to prevent memory overflow during ingestion. After loading, type normalization and indexing were applied to improve filtering and aggregation performance.

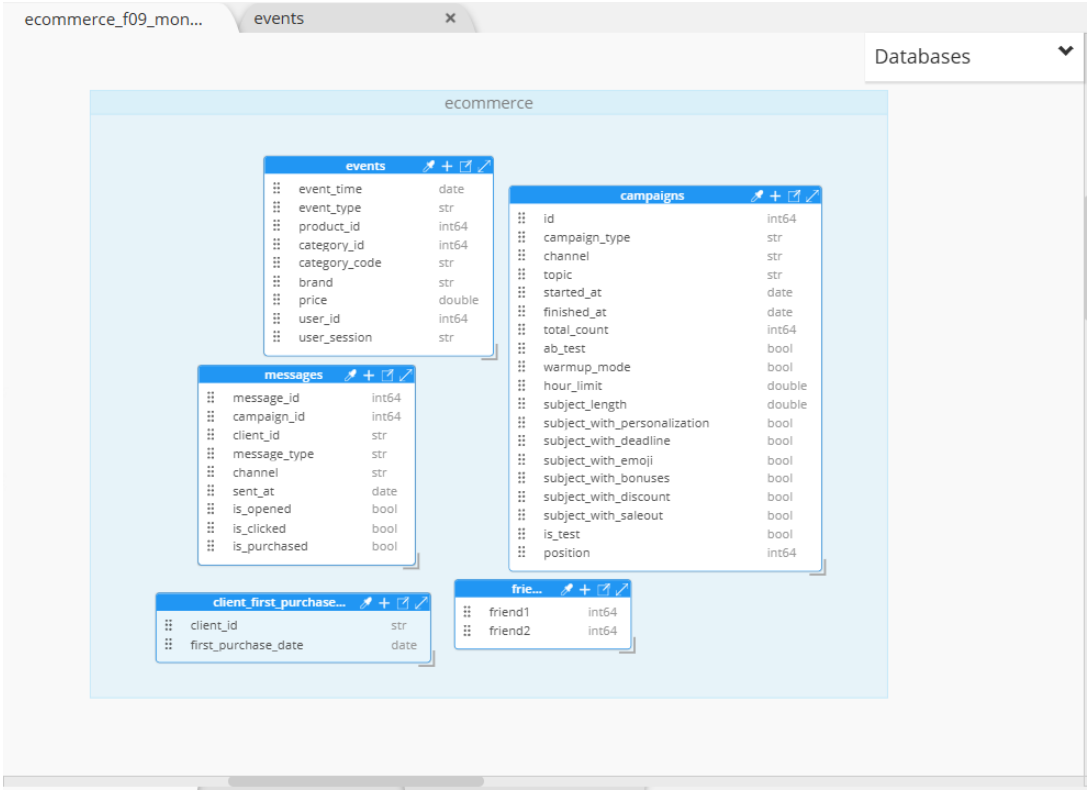

Figure 2: MongoDB document data model created in Hackolade.

1.4 Neo4j Data Model

The Neo4j implementation models the dataset as a graph structure with nodes representing users, products, campaigns, and categories. Relationships such as friendships, interactions, and campaign participation are represented as edges.

This structure allows efficient traversal of social relationships and product interactions.

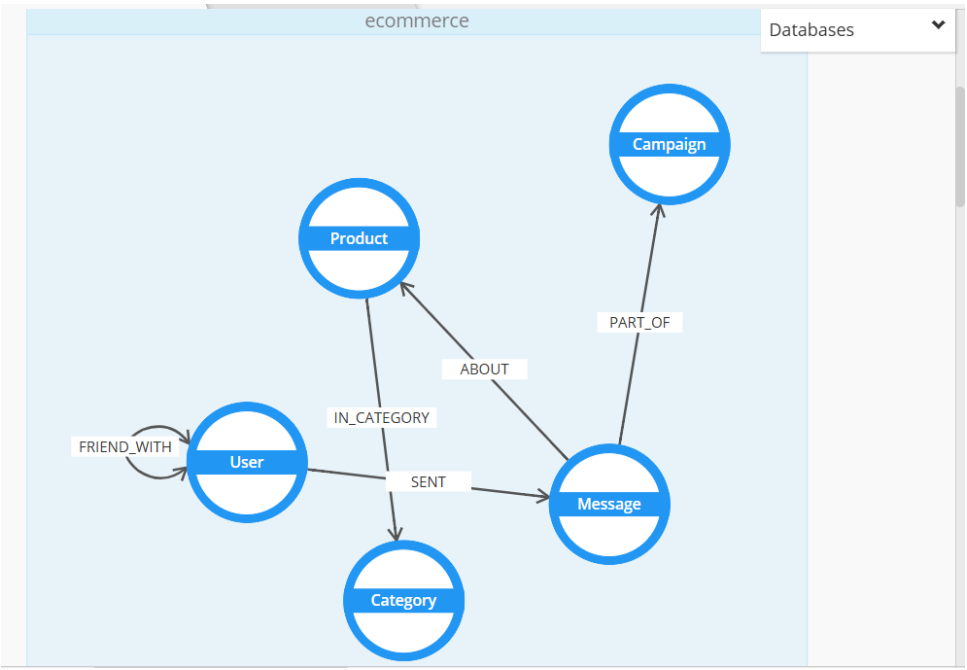

Figure 3: Neo4j graph data model created in Hackolade.

1.5 Custom Scalable Data Model

To improve scalability beyond the three baseline database models, a custom analytical model was designed around the highest-volume dataset component, namely user behavior events.

Raw event data is loaded in batches using Python-based loading scripts, while frequently used analytical structures such as user-product interaction scores, campaign conversion summaries, and friend-influenced recommendation candidates are precomputed and stored in compact derived tables.

The main advantage of this approach is improved scalability. By performing part of the analytical computation during preprocessing, the database queries become significantly lighter and faster because they avoid repeated scans of very large behavioral datasets.

Python-based batch processing also allows flexible data cleaning, normalization, and transformation before the data is inserted into the analytical tables.

However, this design also introduces disadvantages. The system becomes more complex because it requires an additional preprocessing stage. Derived tables introduce redundancy and may require recomputation when raw data changes.

Despite these limitations, separating raw data ingestion from analytical query paths is a common strategy for improving performance in large-scale data processing systems.

1.6 Rationale for the Models

The models were intentionally designed to represent different storage philosophies and scalability approaches.

- PostgreSQL emphasizes structured relational analytics.
- MongoDB emphasizes flexible document storage and chunk-based ingestion.

- Neo4j emphasizes relationship traversal and graph-native queries.
- The custom scalable model emphasizes preprocessing and separation between raw data ingestion and analytical query execution.

2 Experimental Results and Benchmarking

2.1 Q1: Campaign Effectiveness

The first analysis evaluated whether marketing campaigns influenced customers to purchase products.

Results showed that several campaigns achieved conversion rates around 1%, demonstrating that campaign messaging influenced purchasing behavior.

Although the conversion percentages were relatively small, the results still demonstrate that marketing campaigns contributed measurable purchasing activity among users.

2.2 Q2: Personalized Product Recommendation

Personalized recommendations were generated for a target user using friendship relationships and weighted behavioral events.

The event types were weighted as follows:

- purchase = 5
- cart = 3
- view = 1

Products already seen by the user were excluded from the recommendation list.

2.3 Q3: Retrieval Based on Recommended Products

In the conceptual schema the attribute `category_code` could be used for keyword-based retrieval. However, during the final data loading process the runtime analytical schemas used for benchmarking were simplified.

Because the simplified runtime schema did not retain the full `category_code` attribute, the final query retrieved products based on the recommended product identifiers obtained from the recommendation stage.

2.4 Benchmark Environment

The benchmark experiments were executed in the following environment:

- Operating System: Ubuntu 24.04.1 LTS running on WSL2
- CPU: Intel Core i5-4300M @ 2.60 GHz
- Logical CPU Cores: 4

- RAM: 3.8 GB
- Architecture: x86_64

Database software versions used in the experiment:

- PostgreSQL 16.13
- MongoDB Server (with MongoDB Shell 2.7.0)
- Neo4j 4.4.48

2.5 Benchmark Results

Each query was executed five times on each database system. The mean execution time and standard deviation were calculated from the recorded execution times.

Table 1: Benchmark results for Q1, Q2, and Q3 across PostgreSQL, MongoDB, and Neo4j

Database	Query	Mean Time (s)	Std Dev (s)
PostgreSQL	Q1	1.961	1.831
PostgreSQL	Q2	0.121	0.0468
PostgreSQL	Q3	0.0747	0.0156
MongoDB	Q1	20.150	22.450
MongoDB	Q2	2.492	0.772
MongoDB	Q3	0.7562	0.022
Neo4j	Q1	15.663	19.566
Neo4j	Q2	1.492	0.078
Neo4j	Q3	1.370	0.070

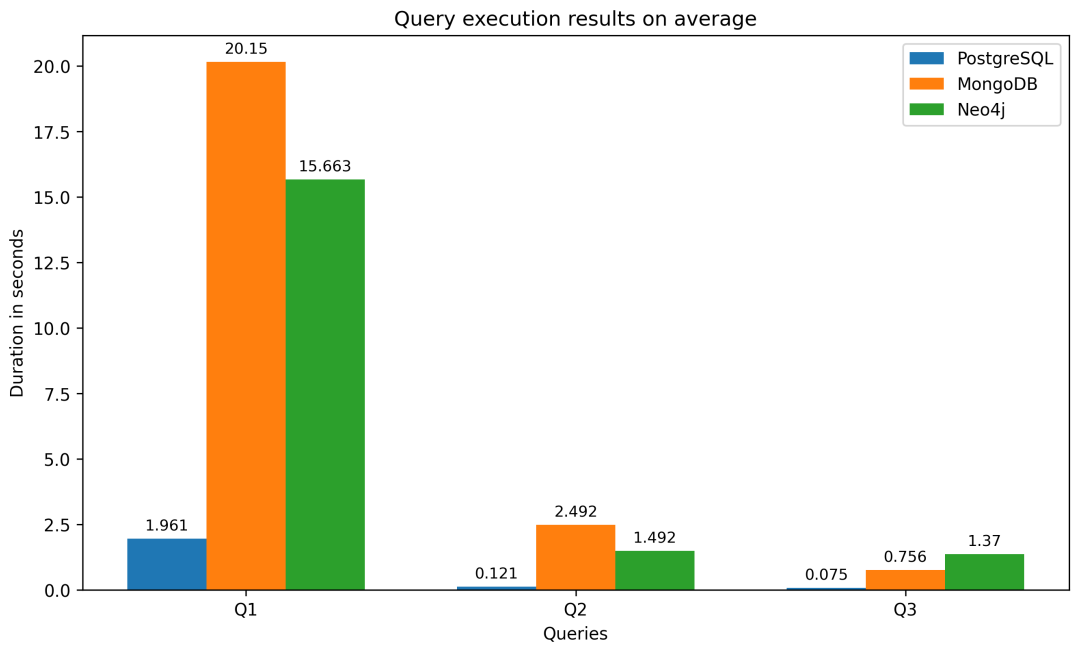

Figure 4: Query execution results on average.

3 Discussion

The benchmark results show that PostgreSQL produced the most stable and fastest execution for aggregation-heavy analytical queries. Its relational optimizer and structured schema allowed efficient aggregation across campaign messages and behavioral datasets.

MongoDB demonstrated strong flexibility during data ingestion and schema management. The document-based structure simplified the loading pipeline and allowed the dataset to be imported in chunks. However, aggregation operations over large collections were slower compared to the relational model.

Neo4j provided the most natural representation of social relationships in the dataset. Friendship connections and recommendation paths are easy to express using graph traversal queries. Although Neo4j did not outperform PostgreSQL in the benchmark results, the graph model provides a clear conceptual advantage for relationship-driven analytics.

These results highlight that no single database model is universally optimal. Instead, the most suitable database architecture depends on the structure of the data, the query workload, and the relationships between entities.

4 Conclusion

This report presented the design, implementation, and evaluation of three database models for an e-commerce dataset using PostgreSQL, MongoDB, and Neo4j.

The benchmark experiments showed that PostgreSQL provided the most stable and fastest execution for the tested analytical queries. MongoDB demonstrated flexibility during data ingestion and schema management, while Neo4j provided a natural representation of social relationships and recommendation paths.

The results confirm that database architecture decisions should be guided by workload characteristics, query complexity, and data relationships rather than relying on a single database paradigm.